ADVANCED DATABASE MANAGEMENT SYSTEMS

ICT3222



Final Report

Group -01

Submitted to - MS. Dilhan

Submitted Date - 26/10/2026

Contents

1. Introduction 4
   1.1 Project Overview and Background 4
   1.2 Problem Statement 4
   1.3 Project Objectives 5
2. Backend solution 6
   2.1 Transaction Management 6
   15.4 Database Security and Data Integrity 6
3. ER diagram 7
4. Relational Mapping 8
5. Table structure 9
6. Architecture of backend 14
7. Tools and technologies 16
8. Hosting backend and reasons for the selection 16
   8.1 Hosting Configuration 16
   8.2 Reasons for Selection 17
9. Security measures for protect the DB 17
10. DB Accounts/Users and the reasons for creating such Accounts/Users 18
   10.1 Database Accounts and Their Purposes 19
   10.2 Reasons for Using Separate Database Accounts 21
   10.3 Database Access Control Implementation 22
15. Code of the DB connection 22
   11.1 Database Connection and Configuration 22
12. Code snippets 24
   12.1. POS Checkout Transaction and Stock Locking 24
   12.2 Stored Procedure for Stock Transfers 25
   12.3 Database Triggers for Stock Alerts and Price History 27
   12.4 Daily Sales Report View 27

13. Problems faced during the development Solutions 28

14. If you are going to host your backend in a cloud environment what are the things/changes that you have to do in your backend 29

14.1. Move the Database to the Cloud 29

14.2. Update Database Connection Settings 29

14.3. Improve Security 29

14.4. Deploy and Monitor the Backend 30

15. What are the possibilities for you to replace your relational DB backend with a non-relational DB technology. 30

15.1. Feasibility of Using NoSQL 30

15.2. Suitable Areas for NoSQL 30

15.3. Challenges of Replacing MySQL 30

15.4. Recommendation 31

1. Introduction

1.1 Project Overview and Background

Supermarkets manage many daily activities, including product sales, stock control, supplier orders, and customer loyalty. Large supermarket businesses may operate several branches in different locations. They need a reliable system to manage information across all branches while allowing each branch to carry out its daily work.

Managing these activities manually billing mistakes, incorrect stock records, and poor customer service. A Multi-Branch Supermarket Management and POS System bring these activities together in one web-based system. It uses a central database to support product management, purchasing, inventory, sales, payments, customer loyalty, staff management, and auditing.

1.2 Problem Statement

Traditional or disconnected supermarket systems can create several problems:

- Stock errors and overselling
  Two or more cashiers may try to sell the same available stock at the same time. Without proper transaction control, stock quantities may become incorrect or negative.

- Product expiry losses
  If products are not tracked by batch and expiry date, expired goods may remain in stock or on shelves.

- Customer loyalty difficulties
  Poor tracking of loyalty points, customer tiers, promotions can cause incorrect rewards and reduce customer satisfaction.

- Weak auditing
  Without reliable records, it may be difficult to identify who changed a product price, adjusted stock, or processed a return.

- Slow reporting
  Large reports that run directly against busy transaction tables may affect the speed of the checkout process.

1.3 Project Objectives

The main objective of this project is to design and develop a reliable database system and web application for managing supermarket operations across multiple branches.

The project aims to:

- Maintain accurate sales, payment, and stock records using MySQL 8.0 and the InnoDB storage engine.

- Use transactions and row-level locking, including SELECT … FOR UPDATE where required, to reduce stock conflicts during simultaneous sales.

- Organize supermarket information in a normalized database covering branches, employees, products, suppliers, inventory, purchasing, sales, customers, loyalty, and auditing.

- Use database objects such as views, stored procedures, functions, triggers, constraints, and scheduled events to support business rules and routine tasks.

- Develop a web application using Next.js and TypeScript, with API routes that connect the website to the MySQL database.

- Improve security through validation, controlled database permissions, password hashing, secure connection settings, and audit records.

2. Backend solution

The backend of the Multi-Branch Supermarket Management and POS System uses MySQL as the database and Next.js with Node.js to handle application operations. The backend manages product details, customers, sales, payments, inventory, loyalty points, and promotions.

- Data storage – Stores supermarket data in related tables.
- Data accuracy – Primary keys and foreign keys maintain correct relationships between tables.
- Transaction support – Transactions help ensure that sales and related updates are completed correctly.
- Database automation – Stored procedures, functions, triggers, and scheduled events support database operations.
- Reporting: - Views help display customer details, sales information, inventory data, and promotion details.

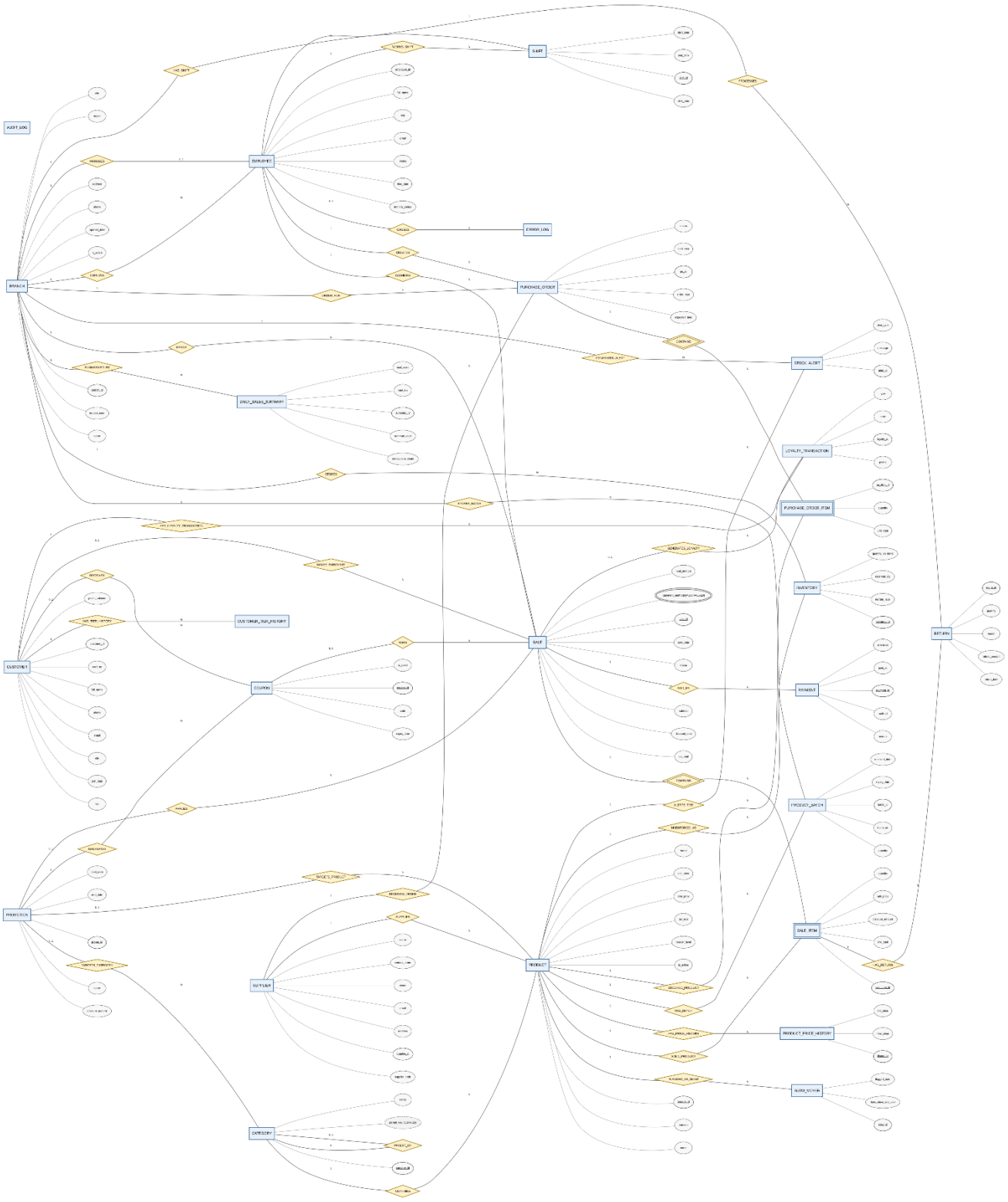
2.1 Transaction Management

The system uses database transactions to maintain data accuracy and consistency. Transactions ensure that related database changes are handled as a single unit. MySQL InnoDB supports COMMIT to save changes and ROLLBACK to undo changes if an error occurs. It also supports row-level locking to manage simultaneous access by multiple users. These features help prevent incorrect data updates and maintain reliable database operations.

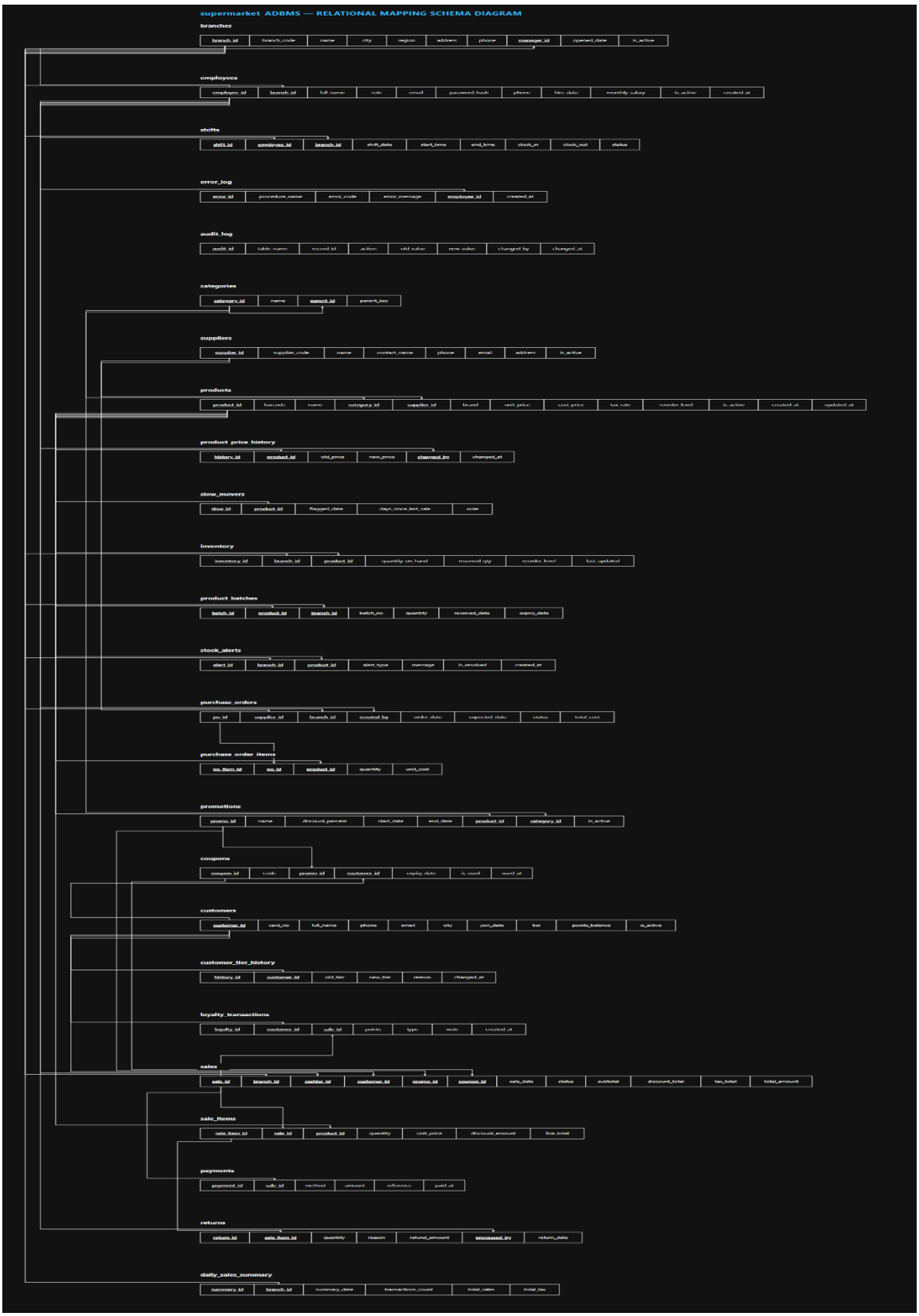
15.4 Database Security and Data Integrity

The system uses database constraints, primary keys, and foreign keys to maintain accurate and reliable data. Input validation and access control help prevent invalid data and unauthorized access. These features help protect database information and maintain the consistency of the supermarket system.

3. ER diagram


AUDIT_LOG
EMPLOYEE
SHIFT
BRANCH
ERROR_LOG
PURCHASE_ORDER
DAILY_SALES_SUMMARY
STOCK_ALERT
LOYALTY_TRANSACTION
PURCHASE_ORDER_ITEM
INVENTORY
RETURN
CUSTOMER
CUSTOMER_TIER_HISTORY
COUPON
SALE
PAYMENT
PRODUCT_BATCH
PROMOTION
SALE_ITEM
SUPPLIER
PRODUCT
PRODUCT_PRICE_HISTORY
CATEGORY

4. Relational Mapping



5. Table structure

1. Table name – Branch

Column	Data type	Key / Constraint
branch_id	INT	PK, AUTO_INCREMENT
branch_code	VARCHAR(10)	UNIQUE, NOT NULL
name	VARCHAR(100)	NOT NULL
city	VARCHAR(60)	NOT NULL
region	VARCHAR(60)	NOT NULL
address	VARCHAR(200)	NULL
phone	VARCHAR(20)	NULL
manager_id	INT	FK→ employees.employee_id, NULL
opened_date	DATE	NOT NULL
is_active	TINYINT(1)	NOT NULL, DEFAULT 1

2. Table name – employees

Column	Data type	Key / Constraint
employee_id	INT	PK, AUTO_INCREMENT
branch_id	INT	FK→branches.branch_id, NOT NULL
full_name	VARCHAR(100)	NOT NULL
role	ENUM	CASHIER, MANAGER, STOCK_CLERK, HR, ADMIN
email	VARCHAR(100)	UNIQUE, NOT NULL
phone	VARCHAR(20)	NULL
hire_date	DATE	NOT NULL
monthly_salary	DECIMAL(10,2)	NOT NULL, CHECK ≥ 0
is_active	TINYINT(1)	NOT NULL, DEFAULT 1
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP

3. Table name – shifts

Column	Data type	Key / Constraint
shift_id	INT	PK, AUTO_INCREMENT
employee_id	INT	FK→employees. Employee_id
branch_id	INT	FK→branches.branch_id
shift_date	DATE	NOT NULL
start_time	TIME	NOT NULL
end_time	TIME	NOT NULL
hire_date	DATE	NOT NULL
clock_in	DATETIME	NULL
clock_out	DATETIME	NULL
status	ENUM	SCHEDULED, PRESENT, ABSENT, COMPLETED; DEFAULT SCHEDULED

4. Table name – audit_log

Column	Data type	Key / Constraint
audit_id	BIGINT	PK, AUTO_INCREMENT
table_name	VARCHAR(64)	NOT NULL
record_id	VARCHAR(64)	NOT NULL
action	ENUM	INSERT, UPDATE, DELETE
old_value	JSON	NULL
new_value	JSON	NULL
changed_by	VARCHAR(100)	NULL
changed_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP

5. Table name – error_log

Column	Data type	Key / Constraint
error_id	BIGINT	PK, AUTO_INCREMENT
procedure_name	VARCHAR(64)	NOT NULL
error_code	VARCHAR(20)	NULL
error_message	VARCHAR(500)	NULL
employee_id	INT	FK→ employees.employee_id, NULL
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP

6. Table name – categories

Column	Data type	Key / Constraint
category_id	INT	PK, AUTO_INCREMENT
name	VARCHAR(80)	NOT NULL
parent_id	INT	FK → categories.category_id, NULL

7.Table name – suppliers

Column	Data type	Key / Constraint
supplier_id	INT	PK, AUTO_INCREMENT
supplier_code	VARCHAR(20)	UNIQUE, NOT NULL
name	VARCHAR(100)	NOT NULL
contact_name	VARCHAR(100)	NULL
phone	VARCHAR(20)	NULL
email	VARCHAR(100)	NULL
address	VARCHAR(200)	NULL
is_active	TINYINT(1)	NOT NULL, DEFAULT 1

8. Table name – slow_movers

Column	Data type	Key / Constraint
slow_id	INT	PK, AUTO_INCREMENT
product_id	INT	FK → products.product_id
flagged_date	DATE	NOT NULL
days_since_last_sale	INT	NOT NULL
note	VARCHAR(200)	NULL

9. Table name – products

Column	Data type	Key / Constraint
product_id	INT	PK, AUTO_INCREMENT
barcode	VARCHAR(20)	UNIQUE, NOT NULL
name	VARCHAR(150)	NOT NULL
category_id	INT	FK→ categories.category_id
supplier_id	INT	FK→ suppliers.supplier_id
brand	VARCHAR(80)	NULL
unit_price	DECIMAL(10,2)	NOT NULL
cost_price	DECIMAL(10,2)	NOT NULL
tax_rate	DECIMAL(5,2)	NOT NULL, DEFAULT 0.00
reorder_level	INT	NOT NULL, DEFAULT 10
is_active	TINYINT(1)	NOT NULL, DEFAULT 1
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP
updated_at	DATETIME	NOT NULL, auto-updated on change

10. Table name –

Column	Data type	Key / Constraint
history_id	INT	PK, AUTO_INCREMENT
product_id	INT	FK → products.product_id
old_price	DECIMAL(10,2)	NOT NULL
new_price	DECIMAL(10,2)	NOT NULL
changed_by	INT	FK → employees.employee_id, NULL
changed_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP

product_price_history

11. Table name – customers

Column	Data type	Key / Constraint
customer_id	INT	PK, AUTO_INCREMENT
card_no	VARCHAR(20)	UNIQUE, NOT NULL
full_name	VARCHAR(100)	NOT NULL
phone	VARCHAR(20)	UNIQUE, NOT NULL
email	VARCHAR(100)	NULL
city	VARCHAR(60)	NULL
join_date	DATE	NOT NULL
tier	ENUM	BRONZE, SILVER, GOLD, PLATINUM; DEFAULT BRONZE
points_balance	INT	NOT NULL, DEFAULT 0, CHECK ≥ 0
is_active	TINYINT(1)	NOT NULL, DEFAULT 1

12. Table name – promotions

Column	Data type	Key / Constraint
promo_id	INT	PK, AUTO_INCREMENT
name	VARCHAR(100)	NOT NULL
discount_percent	DECIMAL(5,2)	NOT NULL, CHECK > 0 and ≤ 100
start_date	DATE	NOT NULL
end_date	DATE	NOT NULL, CHECK ≥ start_date
product_id	INT	FK→ products.product_id, NULL
category_id	INT	FK→ categories.category_id, NULL
is_active	TINYINT(1)	NOT NULL, DEFAULT 1

13.Table name – customer_tier_history

Column	Data type	Key / Constraint
history_id	INT	PK, AUTO_INCREMENT
customer_id	INT	FK→ customers.customer_id
old_tier	ENUM	BRONZE, SILVER, GOLD, PLATINUM; NULL
new_tier	ENUM	BRONZE, SILVER, GOLD, PLATINUM
reason	VARCHAR(200)	NULL
changed_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP

14 .Table name – loyalty_transactions

Column	Data type	Key / Constraint
loyalty_id	INT	PK, AUTO_INCREMENT
customer_id	INT	FK → customers.customer_id
sale_id	INT	FK → sales.sale_id, NULL
points	INT	NOT NULL
type	ENUM	EARN, REDEEM, EXPIRE, ADJUST
note	VARCHAR(200)	NULL
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP

15. Table name – Inventory

Column	Data type	Key / Constraint
inventory_id	INT	PK, AUTO_INCREMENT
branch_id	INT	FK → branches.branch_id
product_id	INT	FK → products.product_id
quantity_on_hand	INT	NOT NULL, DEFAULT 0, CHECK ≥ 0
reserved_qty	INT	NOT NULL, DEFAULT 0, CHECK ≥ 0
reorder_level	INT	NOT NULL, DEFAULT 10
last_updated	DATETIME	NOT NULL, auto-updated on change

16. Table name – coupons

Column	Data type	Key / Constraint
coupon_id	INT	PK, AUTO_INCREMENT
code	VARCHAR(30)	UNIQUE, NOT NULL
promo_id	INT	FK → promotions.promo_id
customer_id	INT	FK → customers.customer_id, NULL
expiry_date	DATE	NOT NULL
is_used	TINYINT(1)	NOT NULL, DEFAULT 0
used_at	DATETIME	NULL

17.Table name - product_batches

Column	Data type	Key / Constraint
batch_id	INT	PK, AUTO_INCREMENT
product_id	INT	FK → products.product_id
branch_id	INT	FK → branches.branch_id
batch_no	VARCHAR(30)	NOT NULL
quantity	INT	NOT NULL, CHECK ≥ 0
received_date	DATE	NOT NULL
expiry_date	DATE	NULL

18. Table name - purchase_orders

Column	Data type	Key / Constraint
po_id	INT	PK, AUTO_INCREMENT
supplier_id	INT	FK → suppliers.supplier_id
branch_id	INT	FK → branches.branch_id
created_by	INT	FK → employees.employee_id
order_date	DATE	NOT NULL
expected_date	DATE	NULL
status	ENUM	DRAFT, SENT, RECEIVED, CANCELLED; DEFAULT DRAFT
total_cost	DECIMAL(12,2)	NOT NULL, DEFAULT 0

19. Table name - purchase_order_items

Column	Data type	Key / Constraint
po_item_id	INT	PK, AUTO_INCREMENT
po_id	INT	FK → purchase_orders.po_id, ON DELETE CASCADE
product_id	INT	FK → products.product_id
quantity	INT	NOT NULL, CHECK > 0
unit_cost	DECIMAL(10,2)	NOT NULL, CHECK > 0

20.Tbale name – stock_alerts

Column	Data type	Key / Constraint
alert_id	INT	PK, AUTO_INCREMENT
branch_id	INT	FK → branches.branch_id
product_id	INT	FK → products.product_id
alert_type	ENUM	LOW_STOCK, OUT_OF_STOCK, EXPIRY
message	VARCHAR(200)	NULL
is_resolved	TINYINT(1)	NOT NULL, DEFAULT 0
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP

21. Table name – sale_items

Column	Data type	Key / Constraint
sale_item_id	INT	PK, AUTO_INCREMENT
sale_id	INT	FK → sales.sale_id, ON DELETE CASCADE
product_id	INT	FK → products.product_id
quantity	INT	NOT NULL, CHECK > 0
unit_price	DECIMAL(10,2)	NOT NULL
discount_amount	DECIMAL(10,2)	NOT NULL, DEFAULT 0
line_total	DECIMAL(12,2)	NOT NULL

22. Table name – Sales

Column	Data type	Key / Constraint
sale_id	INT	PK, AUTO_INCREMENT
branch_id	INT	FK → branches.branch_id
cashier_id	INT	FK → employees.employee_id
customer_id	INT	FK → customers.customer_id, NULL
coupon_id	INT	FK → coupons.coupon_id, NULL
promo_id	INT	FK → promotions.promo_id, NULL
sale_date	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP
status	ENUM	OPEN, COMPLETED, CANCELLED; DEFAULT OPEN
subtotal	DECIMAL(12,2)	NOT NULL, DEFAULT 0
discount_total	DECIMAL(12,2)	NOT NULL, DEFAULT 0
tax_total	DECIMAL(12,2)	NOT NULL, DEFAULT 0
total_amount	DECIMAL(12,2)	NOT NULL, DEFAULT 0

23 . Table name – payments

Column	Data type	Key / Constraint
payment_id	INT	PK, AUTO_INCREMENT
sale_id	INT	FK → sales.sale_id
method	ENUM	CASH, CARD, MOBILE, POINTS
amount	DECIMAL(12,2)	NOT NULL, CHECK > 0
reference	VARCHAR(50)	NULL
paid_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP

24. Table name – returns

Column	Data type	Key / Constraint
return_id	INT	PK, AUTO_INCREMENT
sale_item_id	INT	FK → sale_items.sale_item_id
quantity	INT	NOT NULL, CHECK > 0
reason	VARCHAR(200)	NULL
refund_amount	DECIMAL(10,2)	NOT NULL
processed_by	INT	FK → employees.employee_id
return_date	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP

25. Table name – daily_sales_summary

Column	Data type	Key / Constraint
summary_id	INT	PK, AUTO_INCREMENT
branch_id	INT	FK → branches.branch_id
summary_date	DATE	NOT NULL
transactions_count	INT	NOT NULL, DEFAULT 0
total_sales	DECIMAL(14,2)	NOT NULL, DEFAULT 0
total_tax	DECIMAL(14,2)	NOT NULL, DEFAULT 0

6. Architecture of backend

The system uses a five-layer backend architecture built with Next.js, Node.js, and MySQL with the InnoDB storage engine. Each layer has a separate responsibility, from receiving user requests to storing and managing data. The application uses direct SQL queries to communicate with the database

Layer 1: API and Request Handling

- Uses Next.js API Route Handlers in src/app/api/.
- Receives requests from the web application.
- Processes requests and returns responses to users.

Layer 2: Authentication and Access Control

- Checks user login sessions.
- Controls access according to user roles and permissions.
- Restricts staff access to their assigned branches where required.

Layer 3: Business Logic and Transaction Management

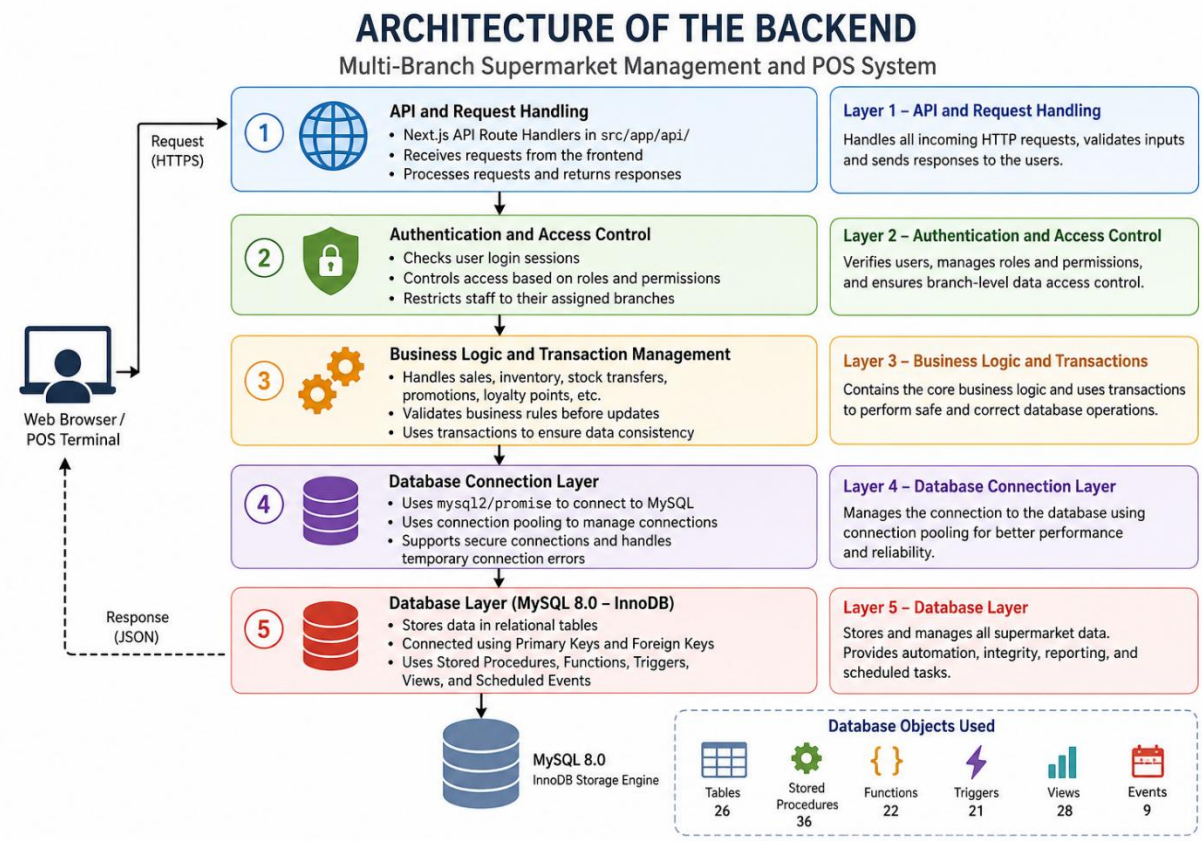
- Handles sales, inventory, stock transfers, promotions, and loyalty points.
- Validates business rules before updating data.
- Uses transactions to maintain data consistency and prevent incorrect stock updates.

Layer 4: Database Connection Layer

- Uses to connect the application to MySQL.
- Uses connection pooling to manage database connections.
- Supports secure database connections and handles selected temporary connection errors.

Layer 5: Database Layer

- Uses MySQL with InnoDB.
- Stores data in relational tables connected through primary and foreign keys.
- Uses stored procedures, functions, triggers, views, and scheduled events to support database operations, automation, and reporting.



7. Tools and technologies

The table describes the main tools and technologies used in the Multi-Branch Supermarket Management and POS System.

Category	Technology	Purpose
Database Tool	MySQL Workbench / Dbeaver	Managing the database and executing SQL queries.
Version Control	Git and GitHub	Managing source code and team collaboration.
Cloud Database Hosting	Aiven Cloud MySQL	Hosts the database online and allows the application to connect remotely.
Backend Framework	Next.js (App Router)	Handles API requests and server-side operations.
Frontend Library	React	Builds the user interface and interactive components.
Programming Language	TypeScript	Provides type checking and helps reduce coding errors.
Database Driver	Database Driver	Database Driver
Database Administration	MySQL Workbench	Creates tables, executes SQL queries, and manages the database.
Data Validation	Zod	Validates user input and API request data before processing.
Authentication	bcryptjs	Hashing passwords before storing them.

8. Hosting backend and reasons for the selection

8.1 Hosting Configuration

The system uses cloud hosting to run the application and store data.

Database Server – Aiven Cloud MySQL 8.0, used to store and manage supermarket data

8.2 Reasons for Selection

- Easy Database Management
  Aiven manages database maintenance and updates, reducing manual work.

- Reliability
  Cloud hosting helps keep the database available and supports recovery from server failures, depending on the selected service plan.

- Good Performance
  MySQL with InnoDB supports efficient data storage, queries, and transactions.

- Data Security
  Encrypted connections help protect customer and financial data when it is transferred between the application and database.

- Scalability
  The application and database resources can be increased as the number of users and amount of data grow.

9. Security measures for protect the DB

The system uses security methods at the application, network, and database levels to protect data and prevent unauthorized access.

Prevention of SQL Injection

- The system uses parameterized SQL queries to prevent SQL injection attacks.
- User inputs are passed separately from SQL commands to prevent users from changing the query structure.

Data Encryption

- SSL/TLS encryption protects data transferred between the Next.js application and the MySQL cloud database.
- It helps protect database credentials and customer information from unauthorized access during transmission.

Least Privilege Access

- The application does not use the MySQL root account to connect to the database.
- Database users are given only the permissions they need. This helps prevent unauthorized changes to the database structure.

Password Hashing and Secret Protection

- User passwords are not stored as plain text.
- Passwords are hashed using bcrypt before being saved in the employees.password_hash column.
- Database credentials are stored in the. Env.local file, which is excluded from Git version control using. Gitignore.

Database Auditing

- The audit_log table records important database changes, including INSERT, UPDATE, and DELETE operations.
- It stores details such as the operator ID, timestamp, and before-and-after data.
- The scheduled event evt_monthly_archive_audit moves audit records older than 90 days to audit_log_archive to help manage the audit data

10.DB Accounts/Users and the reasons for creating such Accounts/Users

The supermarket management system uses separate database accounts to control access to data and improve database security. Each account is given permissions based on the tasks it needs to perform. This helps prevent unauthorized access, accidental data changes, and misuse of database resources.

10.1 Database Accounts and Their Purposes

Assigned Role	Main Permissions	Reason for Creating the Account
Admin	Manage users, database settings, and database operations.	To control the database, manage access permissions, and maintain database security.
Stock clerk	View and update inventory, product batches, and purchase order information	To manage stock levels and track product availability.
Cashier	View products and customers, record sales and payments, and process transactions	To handle customer purchases and payment operations securely.
Marketing manager	View customer and loyalty information, manage promotions and access marketing reports	To manage promotional offers, customer loyalty programs, and marketing activities.
Product manager	View and manage product details, categories, suppliers, and product prices	To maintain accurate product information and manage the product catalog.

```sql
-- ======================================================================
-- 1. CREATE ROLES (IF NOT EXISTS)
-- ======================================================================
CREATE ROLE IF NOT EXISTS 'role_admin';
CREATE ROLE IF NOT EXISTS 'role_catalog_manager';
CREATE ROLE IF NOT EXISTS 'role_stock_clerk';
CREATE ROLE IF NOT EXISTS 'role_cashier';
CREATE ROLE IF NOT EXISTS 'role_marketing_manager';


-- ======================================================================
-- 2. ROLE 1: CATALOG MANAGER
-- ======================================================================
GRANT SELECT, INSERT, UPDATE ON supermarket_ADBMS.products TO 'role_catalog_manager';
GRANT SELECT, INSERT, UPDATE ON supermarket_ADBMS.categories TO 'role_catalog_manager';
GRANT SELECT, INSERT, UPDATE ON supermarket_ADBMS.suppliers TO 'role_catalog_manager';
GRANT SELECT, INSERT         ON supermarket_ADBMS.product_price_history TO 'role_catalog_manager';
GRANT SELECT, INSERT, UPDATE ON supermarket_ADBMS.slow_movers TO 'role_catalog_manager';
GRANT SELECT                 ON supermarket_ADBMS.inventory TO 'role_catalog_manager';

-- Views
GRANT SELECT ON supermarket_ADBMS.v_product_etl TO 'role_catalog_manager';
GRANT SELECT ON supermarket_ADBMS.vw_branch_inventory TO 'role_catalog_manager';

-- Stored Procedures
GRANT EXECUTE ON PROCEDURE supermarket_ADBMS.proc_add_product TO 'role_catalog_manager';
GRANT EXECUTE ON PROCEDURE supermarket_ADBMS.proc_update_price TO 'role_catalog_manager';
GRANT EXECUTE ON PROCEDURE supermarket_ADBMS.proc_discontinue_product TO 'role_catalog_manager';
GRANT EXECUTE ON PROCEDURE supermarket_ADBMS.proc_flag_slow_movers TO 'role_catalog_manager';

-- Functions
GRANT EXECUTE ON FUNCTION supermarket_ADBMS.fn_profit_margin TO 'role_catalog_manager';
GRANT EXECUTE ON FUNCTION supermarket_ADBMS.fn_price_with_tax TO 'role_catalog_manager';
GRANT EXECUTE ON FUNCTION supermarket_ADBMS.fn_category_path TO 'role_catalog_manager';


-- ======================================================================
-- 3. ROLE 2: STOCK CLERK
-- ======================================================================
GRANT SELECT, UPDATE         ON supermarket_ADBMS.inventory TO 'role_stock_clerk';
GRANT SELECT, INSERT, UPDATE ON supermarket_ADBMS.product_batches TO 'role_stock_clerk';
GRANT SELECT, INSERT, UPDATE ON supermarket_ADBMS.purchase_orders TO 'role_stock_clerk';
GRANT SELECT, INSERT, UPDATE ON supermarket_ADBMS.purchase_order_items TO 'role_stock_clerk';
GRANT SELECT, INSERT, UPDATE ON supermarket_ADBMS.stock_alerts TO 'role_stock_clerk'; -- (Fixed: Added INSERT)
GRANT SELECT                 ON supermarket_ADBMS.products TO 'role_stock_clerk';
GRANT SELECT                 ON supermarket_ADBMS.categories TO 'role_stock_clerk';
GRANT SELECT                 ON supermarket_ADBMS.suppliers TO 'role_stock_clerk';
GRANT SELECT                 ON supermarket_ADBMS.branches TO 'role_stock_clerk';
```

```
-- ======================================================================
-- 3. ROLE 2: STOCK CLERK
-- ======================================================================
GRANT SELECT, UPDATE         ON supermarket_ADBMS.inventory TO 'role_stock_clerk';
GRANT SELECT, INSERT, UPDATE ON supermarket_ADBMS.product_batches TO 'role_stock_clerk';
GRANT SELECT, INSERT, UPDATE ON supermarket_ADBMS.purchase_orders TO 'role_stock_clerk';
GRANT SELECT, INSERT, UPDATE ON supermarket_ADBMS.purchase_order_items TO 'role_stock_clerk';
GRANT SELECT, INSERT, UPDATE ON supermarket_ADBMS.stock_alerts TO 'role_stock_clerk'; -- (Fixed: Added INSERT)
GRANT SELECT                 ON supermarket_ADBMS.products TO 'role_stock_clerk';
GRANT SELECT                 ON supermarket_ADBMS.categories TO 'role_stock_clerk';
GRANT SELECT                 ON supermarket_ADBMS.suppliers TO 'role_stock_clerk';
GRANT SELECT                 ON supermarket_ADBMS.branches TO 'role_stock_clerk';

-- Views
GRANT SELECT ON supermarket_ADBMS.vw_branch_inventory TO 'role_stock_clerk';
GRANT SELECT ON supermarket_ADBMS.vw_batch_expiry_status TO 'role_stock_clerk';
GRANT SELECT ON supermarket_ADBMS.vw_purchase_order_details TO 'role_stock_clerk';
GRANT SELECT ON supermarket_ADBMS.vw_purchase_order_summary TO 'role_stock_clerk';
GRANT SELECT ON supermarket_ADBMS.vw_stock_alert_dashboard TO 'role_stock_clerk';

-- Stored Procedures
GRANT EXECUTE ON PROCEDURE supermarket_ADBMS.sp_check_low_stock TO 'role_stock_clerk';
GRANT EXECUTE ON PROCEDURE supermarket_ADBMS.sp_expiring_batches TO 'role_stock_clerk';
GRANT EXECUTE ON PROCEDURE supermarket_ADBMS.sp_update_inventory TO 'role_stock_clerk';
GRANT EXECUTE ON PROCEDURE supermarket_ADBMS.sp_create_purchase_order TO 'role_stock_clerk';
GRANT EXECUTE ON PROCEDURE supermarket_ADBMS.sp_receive_purchase_order TO 'role_stock_clerk';
GRANT EXECUTE ON PROCEDURE supermarket_ADBMS.sp_transfer_stock TO 'role_stock_clerk';
GRANT EXECUTE ON PROCEDURE supermarket_ADBMS.sp_record_stock_disposal TO 'role_stock_clerk';
GRANT EXECUTE ON PROCEDURE supermarket_ADBMS.sp_auto_dispose_expired_batches TO 'role_stock_clerk';

-- Functions
GRANT EXECUTE ON FUNCTION supermarket_ADBMS.fn_available_qty TO 'role_stock_clerk';
GRANT EXECUTE ON FUNCTION supermarket_ADBMS.fn_stock_status TO 'role_stock_clerk';
GRANT EXECUTE ON FUNCTION supermarket_ADBMS.fn_product_sales_velocity TO 'role_stock_clerk';
GRANT EXECUTE ON FUNCTION supermarket_ADBMS.fn_suggested_reorder_qty TO 'role_stock_clerk';
GRANT EXECUTE ON FUNCTION supermarket_ADBMS.fn_days_to_expiry TO 'role_stock_clerk';
GRANT EXECUTE ON FUNCTION supermarket_ADBMS.fn_branch_replenishment_cost TO 'role_stock_clerk';


-- ======================================================================
-- 4. ROLE 3: CASHIER
-- ======================================================================
GRANT SELECT                 ON supermarket_ADBMS.products TO 'role_cashier';
GRANT SELECT, UPDATE         ON supermarket_ADBMS.inventory TO 'role_cashier'; -- (Fixed: Added UPDATE for POS checkout stock deduction)
GRANT SELECT                 ON supermarket_ADBMS.promotions TO 'role_cashier';
GRANT SELECT                 ON supermarket_ADBMS.coupons TO 'role_cashier';
GRANT SELECT, UPDATE         ON supermarket_ADBMS.customers TO 'role_cashier'; -- (Fixed: Added UPDATE for points redemption)
GRANT SELECT, INSERT         ON supermarket_ADBMS.sales TO 'role_cashier';
GRANT SELECT, INSERT         ON supermarket_ADBMS.sale_items TO 'role_cashier';
GRANT SELECT, INSERT         ON supermarket_ADBMS.payments TO 'role_cashier';
GRANT SELECT, INSERT         ON supermarket_ADBMS.returns TO 'role_cashier';
```

You have Docker installed on you
install the recommended extensi

```
-- Views
GRANT SELECT ON supermarket_ADBMS.sale_details_vw TO 'role_cashier';
GRANT SELECT ON supermarket_ADBMS.daily_sales_vw TO 'role_cashier';
GRANT SELECT ON supermarket_ADBMS.cashier_sales_vw TO 'role_cashier';
GRANT SELECT ON supermarket_ADBMS.payment_summary_vw TO 'role_cashier';

-- Stored Procedures
GRANT EXECUTE ON PROCEDURE supermarket_ADBMS.CreateSale TO 'role_cashier';
GRANT EXECUTE ON PROCEDURE supermarket_ADBMS.AddSaleItem TO 'role_cashier';
GRANT EXECUTE ON PROCEDURE supermarket_ADBMS.UpdateSaleTotals TO 'role_cashier';
GRANT EXECUTE ON PROCEDURE supermarket_ADBMS.CompleteSale TO 'role_cashier';
GRANT EXECUTE ON PROCEDURE supermarket_ADBMS.ProcessReturn TO 'role_cashier';

-- Functions
GRANT EXECUTE ON FUNCTION supermarket_ADBMS.CalculateLineTotal TO 'role_cashier';
GRANT EXECUTE ON FUNCTION supermarket_ADBMS.GetTotalPaid TO 'role_cashier';
GRANT EXECUTE ON FUNCTION supermarket_ADBMS.GetRemainingAmount TO 'role_cashier';
GRANT EXECUTE ON FUNCTION supermarket_ADBMS.GetSaleDiscount TO 'role_cashier';


-- ======================================================================
-- 5. ROLE 4: MARKETING MANAGER
-- ======================================================================
GRANT SELECT, INSERT, UPDATE ON supermarket_ADBMS.promotions TO 'role_marketing_manager';
GRANT SELECT, INSERT, UPDATE ON supermarket_ADBMS.coupons TO 'role_marketing_manager';
GRANT SELECT, INSERT, UPDATE ON supermarket_ADBMS.loyalty_transactions TO 'role_marketing_manager';
GRANT SELECT, INSERT, UPDATE ON supermarket_ADBMS.customer_tier_history TO 'role_marketing_manager';
GRANT SELECT, INSERT, UPDATE ON supermarket_ADBMS.customers TO 'role_marketing_manager';
GRANT SELECT                 ON supermarket_ADBMS.products TO 'role_marketing_manager';
GRANT SELECT                 ON supermarket_ADBMS.categories TO 'role_marketing_manager';

-- Views
GRANT SELECT ON supermarket_ADBMS.customer_etl_vw TO 'role_marketing_manager';
GRANT SELECT ON supermarket_ADBMS.active_promotions_vw TO 'role_marketing_manager';
GRANT SELECT ON supermarket_ADBMS.customer_summary_vw TO 'role_marketing_manager';
GRANT SELECT ON supermarket_ADBMS.top_customers_vw TO 'role_marketing_manager';

-- Stored Procedures
GRANT EXECUTE ON PROCEDURE supermarket_ADBMS.register_customer TO 'role_marketing_manager';
GRANT EXECUTE ON PROCEDURE supermarket_ADBMS.redeem_points TO 'role_marketing_manager';
GRANT EXECUTE ON PROCEDURE supermarket_ADBMS.add_ltpoints TO 'role_marketing_manager';
GRANT EXECUTE ON PROCEDURE supermarket_ADBMS.create_promotion TO 'role_marketing_manager';
GRANT EXECUTE ON PROCEDURE supermarket_ADBMS.recalculate_tiers TO 'role_marketing_manager';
GRANT EXECUTE ON PROCEDURE supermarket_ADBMS.proc_apply_coupon TO 'role_marketing_manager';
GRANT EXECUTE ON PROCEDURE supermarket_ADBMS.get_active_promotions TO 'role_marketing_manager';
GRANT EXECUTE ON PROCEDURE supermarket_ADBMS.get_deactive_promotions TO 'role_marketing_manager';
GRANT EXECUTE ON PROCEDURE supermarket_ADBMS.proc_assign_top10_coupons TO 'role_marketing_manager';
GRANT EXECUTE ON PROCEDURE supermarket_ADBMS.sp_top_10_loyalty_customers_all_branches TO 'role_marketing_manager';
GRANT EXECUTE ON PROCEDURE supermarket_ADBMS.sp_top_10_loyalty_customers_per_branch TO 'role_marketing_manager';
```

```
-- Functions
GRANT EXECUTE ON FUNCTION supermarket_ADBMS.lt_points_earned TO 'role_marketing_manager';
GRANT EXECUTE ON FUNCTION supermarket_ADBMS.get_discount TO 'role_marketing_manager';
GRANT EXECUTE ON FUNCTION supermarket_ADBMS.modify_customer_tier TO 'role_marketing_manager';
GRANT EXECUTE ON FUNCTION supermarket_ADBMS.promo_isactive TO 'role_marketing_manager';


-- ====================================================================
-- 6. ROLE 5: ADMIN
-- ====================================================================
GRANT ALL PRIVILEGES ON supermarket_ADBMS.* TO 'role_admin' WITH GRANT OPTION;


-- ====================================================================
-- 7. CREATE DATABASE USER ACCOUNTS
-- ====================================================================
CREATE USER IF NOT EXISTS 'admin_user'@'%'          IDENTIFIED BY 'Admin#1234';
CREATE USER IF NOT EXISTS 'catalog_mgr_user'@'%'   IDENTIFIED BY 'Catalog#1234';
CREATE USER IF NOT EXISTS 'stock_clerk_user'@'%'   IDENTIFIED BY 'Stock#1234';
CREATE USER IF NOT EXISTS 'cashier_user'@'%'       IDENTIFIED BY 'Cashier#1234';
CREATE USER IF NOT EXISTS 'marketing_user'@'%'     IDENTIFIED BY 'Marketing#1234';


-- ====================================================================
-- 8. ASSIGN ROLES TO USERS & ACTIVATE AS DEFAULT
-- ====================================================================


GRANT 'role_admin'             TO 'admin_user'@'%';
GRANT 'role_catalog_manager'   TO 'catalog_mgr_user'@'%';
GRANT 'role_stock_clerk'       TO 'stock_clerk_user'@'%';
GRANT 'role_cashier'           TO 'cashier_user'@'%';
GRANT 'role_marketing_manager' TO 'marketing_user'@'%';

SET DEFAULT ROLE 'role_admin'             TO 'admin_user'@'%';
SET DEFAULT ROLE 'role_catalog_manager'   TO 'catalog_mgr_user'@'%';
SET DEFAULT ROLE 'role_stock_clerk'       TO 'stock_clerk_user'@'%';
SET DEFAULT ROLE 'role_cashier'           TO 'cashier_user'@'%';
SET DEFAULT ROLE 'role_marketing_manager' TO 'marketing_user'@'%';

FLUSH PRIVILEGES;
```

10.2 Reasons for Using Separate Database Accounts

1. Improved Security **–** Each user receives only the permissions needed for their work.
2. Prevention of Unauthorized Changes – Restricted permissions help prevent users from changing or deleting data they do not manage.
3. Protection of Customer and Sales Data – Inventory staff and reporting users do not automatically receive full access to all database information.
4. Better Database Management – The database administrator can manage users, permissions, and database changes separately from normal application operations.
5. Safe POS Operations – Cashiers can access product information and process sales without receiving full database administration privileges.

10.3 Database Access Control Implementation

The system uses MySQL Data Control Language (DCL) commands, such as CREATE USER and GRANT, to create database accounts and assign permissions.

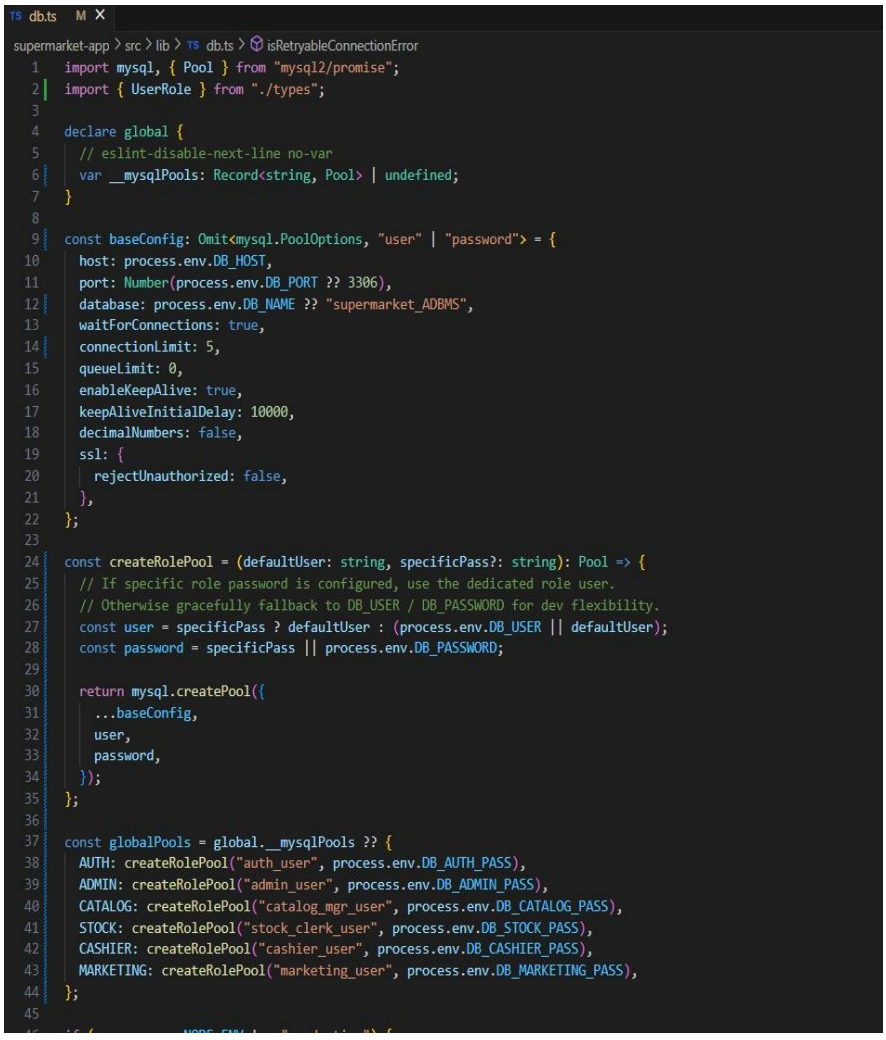
For example, the analytics_user account receives read-only access to selected reporting views, while the cashier_user account receives permissions needed for POS operations. The supermarket_db account is reserved for database administration.

These separate accounts support secure database access, protect important information, and help each project member perform their assigned responsibilities.

15.Code of the DB connection

The database connection is managed using the mysql2/promise library in the src/lib/db.ts file.

11.1 Database Connection and Configuration



```ts
if (process.env.NODE_ENV !== "production") {
  global.__mysqlPools = globalPools;
}

export const pools = globalPools;

export type DbPoolKey = "AUTH" | "ADMIN" | "CATALOG" | "STOCK" | "CASHIER" | "MARKETING";

/**
 * Returns the MySQL connection pool tailored to the user's role or 'AUTH'.
 * Enforces role-level database privileges at the MySQL layer.
 */
export function getPool(role?: UserRole | DbPoolKey | "AUTH"): Pool {
  if (!role) {
    return pools.ADMIN;
  }
  if (role === "AUTH") {
    return pools.AUTH;
  }
  if (role === "ADMIN") {
    return pools.ADMIN;
  }
  if (role === "CASHIER") {
    return pools.CASHIER;
  }
  if (role === "STOCK_CLERK" || role === "STOCK") {
    return pools.STOCK;
  }
  if (role === "CATALOG_MANAGER" || role === "CATALOG") {
    return pools.CATALOG;
  }
  if (role === "MARKETING") {
    return pools.MARKETING;
  }
  return pools.ADMIN;
}

// Backward-compatible default pool
export const pool: Pool = pools.ADMIN;
export default pool;

function isRetryableConnectionError(err: unknown): boolean {
  if (!err || typeof err !== "object") {
    return false;
  }
```

```ts
    error.code === "ECONNRESET" ||
    error.code === "PROTOCOL_CONNECTION_LOST" ||
    error.message?.includes("ECONNRESET") === true
  );
}

function isReadOnlyQuery(sql: string): boolean {
  return /^\s*SELECT\b/i.test(sql);
}

export async function query<T = any>(
  sql: string,
  params: any[] = [],
  role?: UserRole | DbPoolKey | "AUTH"
): Promise<T[]> {
  const targetPool = getPool(role);
  try {
    const [rows] = await targetPool.query(sql, params);
    return rows as T[];
  } catch (err: unknown) {
    // A write may have committed before a connection failure was reported.
    // Retrying it here could duplicate inventory, receipts, or audit records.
    if (isRetryableConnectionError(err) && isReadOnlyQuery(sql)) {
      const [rows] = await targetPool.query(sql, params);
      return rows as T[];
    }
    throw err;
  }
}
```

```ts
export async function queryOne<T = any>(
  sql: string,
  params: any[] = [],
  role?: UserRole | DbPoolKey | "AUTH"
): Promise<T | null> {
  const rows = await query<T>(sql, params, role);
  return rows.length > 0 ? rows[0] : null;
}

export async function withTransaction<T>(
  callback: (conn: mysql.PoolConnection) => Promise<T>,
  role?: UserRole | DbPoolKey
): Promise<T> {
  const conn = await getPool(role).getConnection();
  try {
    await conn.beginTransaction();
    const result = await callback(conn);
    await conn.commit();
    return result;
  } catch (error) {
    await conn.rollback();
    throw error;
  } finally {
    conn.release();
  }
}
```

12.Code snippets

This shows the main code used in the Multi-Branch Supermarket Management and POS System.

12.1. POS Checkout Transaction and Stock Locking

File – src/app/api/pos/checkout/route.ts

This code handles the checkout process when a customer buys products. It saves the sale, updates stock, records sale items, and adds an audit log. A database transaction helps keep all these changes together. The FOR-UPDATE statement locks selected rows to prevent two users from changing the same stock at the same time.



```
const connection = await pool.getConnection();

try {
  await connection.beginTransaction();

  let subtotal = 0;
  let discount_total = 0;
  let tax_total = 0;
  const validatedItems: any[] = [];

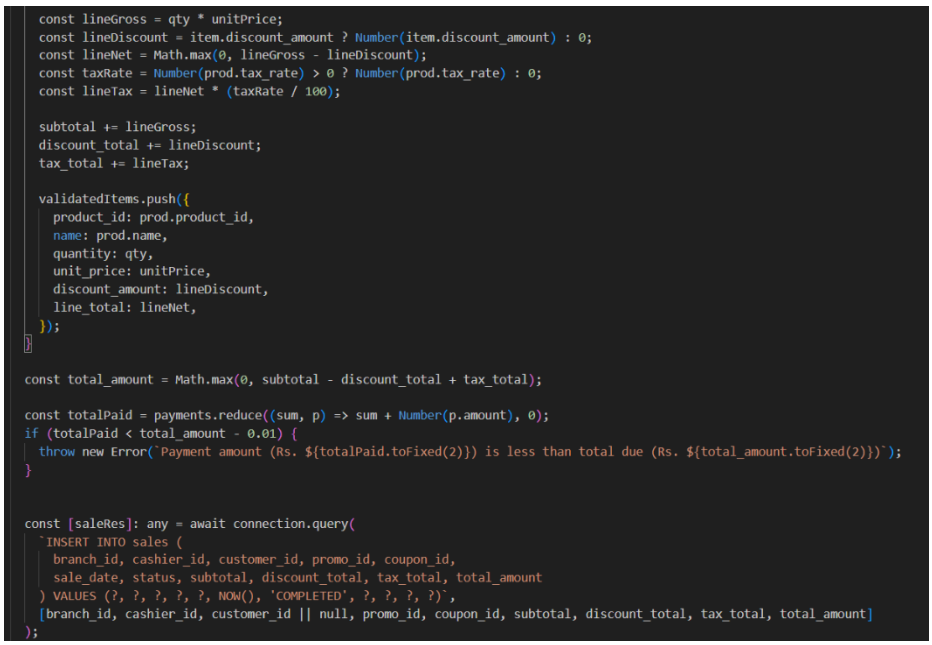

  for (const item of items) {
    const [prodRows]: any = await connection.query(
      "SELECT product_id, barcode, name, unit_price, cost_price, tax_rate, is_active FROM products WHERE product_id = ? FOR UPDATE",
      [item.product_id]
    );

    if (prodRows.length === 0 || prodRows[0].is_active !== 1) {
      throw new Error(`Product ID ${item.product_id} is unavailable or inactive.`);
    }

    const prod = prodRows[0];
    const unitPrice = Number(prod.unit_price);
    const qty = item.quantity;


    const [invRows]: any = await connection.query(
      "SELECT inventory_id, quantity_on_hand FROM inventory WHERE branch_id = ? AND product_id = ? FOR UPDATE",
      [branch_id, item.product_id]
    );

    const currentStock = invRows.length > 0 ? Number(invRows[0].quantity_on_hand) : 0;
    if (currentStock < qty) {
      throw new Error(`Insufficient stock for "${prod.name}" (Requested: ${qty}, In Stock: ${currentStock})`);
    }
```



```
    const lineGross = qty * unitPrice;
    const lineDiscount = item.discount_amount ? Number(item.discount_amount) : 0;
    const lineNet = Math.max(0, lineGross - lineDiscount);
    const taxRate = Number(prod.tax_rate) > 0 ? Number(prod.tax_rate) : 0;
    const lineTax = lineNet * (taxRate / 100);

    subtotal += lineGross;
    discount_total += lineDiscount;
    tax_total += lineTax;

    validatedItems.push({
      product_id: prod.product_id,
      name: prod.name,
      quantity: qty,
      unit_price: unitPrice,
      discount_amount: lineDiscount,
      line_total: lineNet,
    });
  }

  const total_amount = Math.max(0, subtotal - discount_total + tax_total);

  const totalPaid = payments.reduce((sum, p) => sum + Number(p.amount), 0);
  if (totalPaid < total_amount - 0.01) {
    throw new Error(`Payment amount (Rs. ${totalPaid.toFixed(2)}) is less than total due (Rs. ${total_amount.toFixed(2)})`);
  }


  const [saleRes]: any = await connection.query(
    `INSERT INTO sales (
      branch_id, cashier_id, customer_id, promo_id, coupon_id,
      sale_date, status, subtotal, discount_total, tax_total, total_amount
    ) VALUES (?, ?, ?, ?, ?, NOW(), 'COMPLETED', ?, ?, ?, ?)`,
    [branch_id, cashier_id, customer_id || null, promo_id, coupon_id, subtotal, discount_total, tax_total, total_amount]
  );
```

```
  const sale_id = saleRes.insertId;


  for (const item of validatedItems) {
    await connection.query(
      `INSERT INTO sale_items (sale_id, product_id, quantity, unit_price, discount_amount, line_total)
       VALUES (?, ?, ?, ?, ?, ?)`,
      [sale_id, item.product_id, item.quantity, item.unit_price, item.discount_amount, item.line_total]
    );


    await connection.query(
      `UPDATE inventory
       SET quantity_on_hand = quantity_on_hand - ?, last_updated = NOW()
       WHERE branch_id = ? AND product_id = ?`,
      [item.quantity, branch_id, item.product_id]
    );


    const [batches]: any = await connection.query(
      `SELECT batch_id, quantity
       FROM product_batches
       WHERE branch_id = ? AND product_id = ? AND quantity > 0
       ORDER BY expiry_date ASC, received_date ASC FOR UPDATE`,
      [branch_id, item.product_id]
    );

    let remainingToDeduct = item.quantity;
    for (const b of batches) {
      if (remainingToDeduct <= 0) break;
      const deductFromBatch = Math.min(b.quantity, remainingToDeduct);
      await connection.query(
        "UPDATE product_batches SET quantity = quantity - ? WHERE batch_id = ?",
        [deductFromBatch, b.batch_id]
      );
      remainingToDeduct -= deductFromBatch;
    }
  }


  await connection.query(
    `INSERT INTO audit_log (table_name, record_id, action, new_value, changed_by, changed_at)
     VALUES ('sales', ?, 'INSERT', ?, NOW())`,
    [String(sale_id), JSON.stringify({ sale_id, branch_id, total_amount }), `cashier_${cashier_id}`]
  );


  await connection.commit();
} catch (err) {

  await connection.rollback()
  throw err;
} finally {

  connection.release();
```

12.2 Stored Procedure for Stock Transfers

File – stock_clerk.sql and src/app/api/inventory/transfers/route.ts

This stored procedure transfers products from one branch to another. It checks the available stock, reduces the quantity at the source branch, increases the quantity at the destination branch, and records the transfer in the audit log. If an SQL error occurs, the transaction is rolled back

```sql
DELIMITER $$

CREATE PROCEDURE sp_transfer_stock(
    IN p_from_branch_id INT,
    IN p_to_branch_id INT,
    IN p_product_id INT,
    IN p_quantity INT,
    IN p_employee_id INT
)
proc_label: BEGIN
    DECLARE v_current_stock INT DEFAULT 0;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;

    -- Validate source and target branches
    IF p_from_branch_id = p_to_branch_id THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Source and destination branch cannot be the same.';
    END IF;

    -- Lock source inventory row exclusively
    SELECT quantity_on_hand INTO v_current_stock
    FROM inventory
    WHERE branch_id = p_from_branch_id AND product_id = p_product_id
    FOR UPDATE;

    IF v_current_stock < p_quantity THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Insufficient stock at source branch for transfer.';
    END IF;

    -- 1. Deduct from source branch
    UPDATE inventory
    SET quantity_on_hand = quantity_on_hand - p_quantity,
        last_updated = NOW()
    WHERE branch_id = p_from_branch_id AND product_id = p_product_id;

    -- 2. Increment destination branch (with upsert)
    INSERT INTO inventory (branch_id, product_id, quantity_on_hand, reserved_qty, reorder_level, last_updated)
    VALUES (p_to_branch_id, p_product_id, p_quantity, 0, 10, NOW())
    ON DUPLICATE KEY UPDATE
        quantity_on_hand = quantity_on_hand + VALUES(quantity_on_hand),
        last_updated = NOW();

    -- 3. Log into central audit trail
    INSERT INTO audit_log (table_name, record_id, action, new_value, changed_by, changed_at)
    VALUES (
        'inventory',
        CONCAT('transfer_', p_product_id),
        'UPDATE',
        JSON_OBJECT(
            'event', 'INTER_BRANCH_TRANSFER',
            'from_branch_id', p_from_branch_id,
            'to_branch_id', p_to_branch_id,
            'product_id', p_product_id,
            'quantity', p_quantity
        ),
        CONCAT('employee_', p_employee_id),
        NOW()
    );

    COMMIT;
END$$

DELIMITER ;
```

12.3 Database Triggers for Stock Alerts and Price History

```
-- Trigger 1: Automatically generate low stock alerts when quantity falls below reorder point
DELIMITER $$
CREATE TRIGGER trg_inventory_after_update
AFTER UPDATE ON inventory
FOR EACH ROW
BEGIN
    IF NEW.quantity_on_hand <= NEW.reorder_level AND OLD.quantity_on_hand > NEW.reorder_level THEN
        INSERT INTO stock_alerts (branch_id, product_id, alert_type, current_quantity, threshold_quantity, cre
        VALUES (NEW.branch_id, NEW.product_id, 'LOW_STOCK', NEW.quantity_on_hand, NEW.reorder_level, NOW())
        ON DUPLICATE KEY UPDATE
            current_quantity = NEW.quantity_on_hand,
            created_at = NOW();
    END IF;
END$$
DELIMITER ;

-- Trigger 2: Maintain immutable price revision audit log
DELIMITER $$
CREATE TRIGGER trg_products_au_price_history
AFTER UPDATE ON products
FOR EACH ROW
BEGIN
    IF OLD.unit_price <> NEW.unit_price OR OLD.cost_price <> NEW.cost_price THEN
        INSERT INTO product_price_history (
            product_id, old_unit_price, new_unit_price,
            old_cost_price, new_cost_price, changed_at
        ) VALUES (
            NEW.product_id, OLD.unit_price, NEW.unit_price,
            OLD.cost_price, NEW.cost_price, NOW()
        );
    END IF;
END$$
DELIMITER ;
```

12.4 Daily Sales Report View

File – sandeepa.sql

```
CREATE OR REPLACE VIEW daily_sales_vw AS
SELECT
    DATE(s.sale_date) AS report_date,
    s.branch_id,
    b.name AS branch_name,
    COUNT(s.sale_id) AS total_orders,
    SUM(s.subtotal) AS gross_sales,
    SUM(s.discount_total) AS total_discounts,
    SUM(s.tax_total) AS total_tax,
    SUM(s.total_amount) AS net_revenue,
    ROUND(AVG(s.total_amount), 2) AS average_order_value
FROM sales s
JOIN branches b ON s.branch_id = b.branch_id
WHERE s.status = 'COMPLETED'
GROUP BY DATE(s.sale_date), s.branch_id, b.name;

```

13. Problems faced during the development Solutions

During the development of the Supermarket Management System, the team faced several database and application problems. These problems were solved by using database transactions, row locking, connection pooling, stored procedures, triggers, and error handling.

15. Stock Errors During Checkout

When two cashiers sold the same product at the same time, the system could show incorrect stock quantities or allow more items to be sold than available.

Solution – The team used SELECT … FOR UPDATE to lock inventory records during checkout. A database constraint was also added to prevent negative stock quantities.

2. Too Many Database Connections

During development, Next.js sometimes created new database connection pools after code changes. This caused connection errors.

Solution – The team stored the connection pool in a global variable and reused it. The number of connections was limited, and connections were released after use.

3. Incomplete Sales Transactions

If an error occurred during checkout, some records could be saved while other records were not. This could cause incorrect sales and stock records.

Solution – The team used database transactions with COMMIT and ROLLBACK. If an error occurred, the transaction was rolled back to prevent incomplete changes.

4.Incorrect Product Batch Selection

Products had different expiry dates. If newer batches were sold first, older products could expire without being sold.

Solution -The team used the First-In, First-Out (FIFO) method to deduct stock from batches with earlier expiry dates first. The system also included procedures to manage expired products.

14. If you are going to host your backend in a cloud environment what are the things/changes that you have to do in your backend

If the Supermarket Management System is hosted in a cloud environment, some changes must be made to the backend. These changes help the system work securely, handle more users, and maintain a reliable connection with the database.

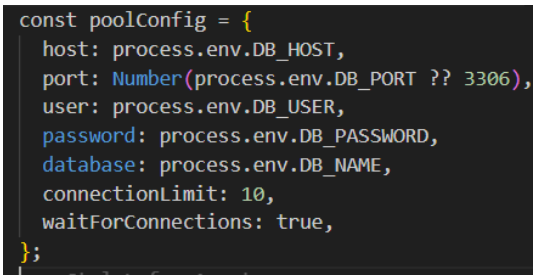
14.1. Move the Database to the Cloud

Move the local MySQL database to a cloud database service, such as Amazon RDS or Google Cloud SQL.

Reason: Cloud databases provide backup and recovery options and allow the backend to connect to the database remotely.

14.2. Update Database Connection Settings

Update src/lib/db.ts to use cloud database credentials through environment variables.



```
const poolConfig = {
  host: process.env.DB_HOST,
  port: Number(process.env.DB_PORT ?? 3306),
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  connectionLimit: 10,
  waitForConnections: true,
};
```

Reason: This allows the backend to connect to the cloud database and manage connections efficiently.

14.3. Improve Security

Change: Enable SSL/TLS with certificate validation and store database passwords and secret keys securely using environment variables or a cloud secrets manager.
Reason: These measures protect database communication and prevent sensitive information from being exposed in source code.

14.4. Deploy and Monitor the Backend

Change: Deploy the Next.js backend to a cloud hosting platform, configure HTTPS, and add error logging and a health-check endpoint such as /api/health.

Reason: Cloud deployment allows users to access the system online, while logging and health checks help identify errors and monitor backend availability.

15. What are the possibilities for you to replace your relational DB backend with a non-relational DB technology.

15.1. Feasibility of Using NoSQL

The current supermarket system uses MySQL, which supports transactions, foreign keys, and complex queries. Replacing it completely with a NoSQL database such as MongoDB would require major changes to the backend and database logic.

15.2. Suitable Areas for NoSQL

NoSQL databases could be used for selected features:

- Product Catalog: MongoDB can store product details with flexible attributes.
- Active Carts and Sessions: Redis can store temporary checkout carts and session data for faster access.
- Audit Logs: Elasticsearch can support searching large volumes of system logs.

15.3. Challenges of Replacing MySQL

The POS system updates sales, payments, and inventory together. MySQL supports these operations through transactions and data integrity constraints. Replacing it completely with NoSQL could make these operations more complex and require rewriting existing database procedures, triggers, and queries.

15.4. Recommendation

A complete replacement of MySQL is not recommended for this supermarket system. The better option is to keep MySQL for sales, payments, and inventory, while adding NoSQL technologies such as MongoDB or Redis only where they provide clear benefits. This approach can improve flexibility and performance without replacing the existing relational database.

Group -01

Team members-

TG/2022/1350 - W.A.K. Imalka

TG/2022/1365 - K.P.G.S. Sandaruwan

TG/2022/1366 - R.A.S. Lakshan

TG/2022/1367 - S.A.P.T. Samarathunga

TG/2022/1392 – A.M.K.S. Amarakoon